

Concurrency

Module 4

Synchronization

- A cooperating process is one that can affect or be affected by other processes executing in the system.
- Cooperating processes can either directly share a logical address space or be allowed to share data only through shared memory or message passing.
- Concurrent access to shared data may result in data inconsistency

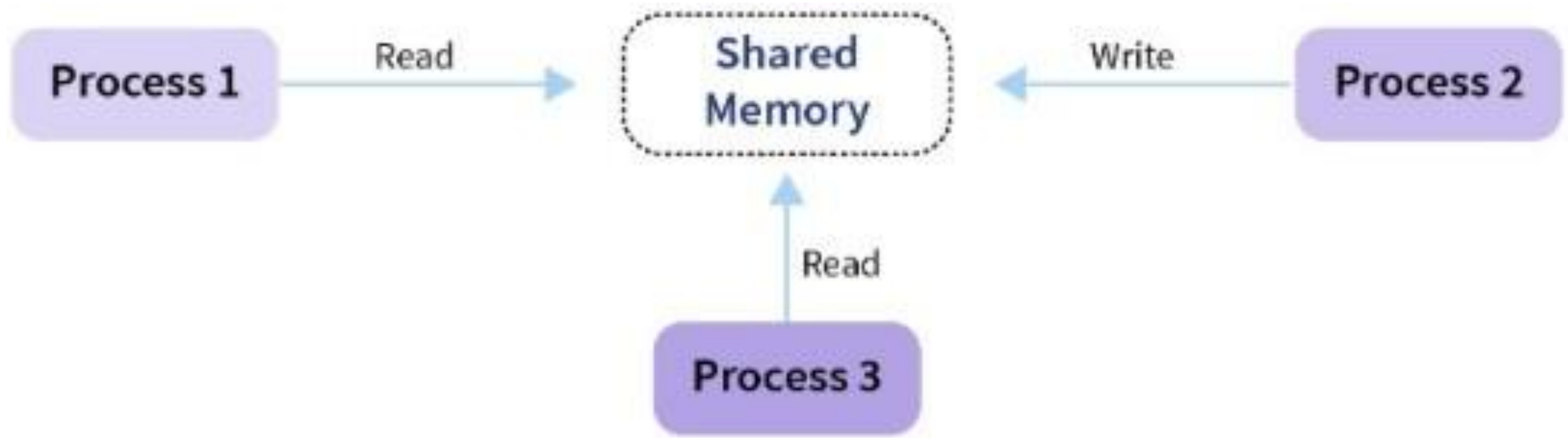
Synchronization - Objective

- The main objective of process synchronization is to
- Ensure that **multiple processes access shared resources** without interfering with each other and
- To prevent the **possibility of inconsistent data** due to concurrent access.

Synchronization

- It is the way by which processes that share the same memory space are managed in an operating system.
- It helps maintain the consistency of data by using variables or hardware so that only one process can make changes to the shared memory at a time.
- Inconsistency of data can occur when various processes share a common resource in a system which is why there is a need for process synchronization in the operating system.

Synchronization



Synchronization

- If a process1 is trying to read the data present in a memory location while another process2 is trying to change the data present at the same location, there is a high chance that the data read by the process1 will be incorrect.

Synchronization- Producer Consumer Problem

$register_1 = count$
 $register_1 = register_1 + 1$
 $count = register_1$

$register_2 = count$
 $register_2 = register_2 - 1$
 $count = register_2$

T_0 :	producer	execute	$register_1 = count$	$\{register_1 = 5\}$
T_1 :	producer	execute	$register_1 = register_1 + 1$	$\{register_1 = 6\}$
T_2 :	consumer	execute	$register_2 = count$	$\{register_2 = 5\}$
T_3 :	consumer	execute	$register_2 = register_2 - 1$	$\{register_2 = 4\}$
T_4 :	producer	execute	$count = register_1$	$\{count = 6\}$
T_5 :	consumer	execute	$count = register_2$	$\{count = 4\}$

Synchronization - Producer Consumer Problem

Since both processes manipulate the variable count concurrently, a situation like this occurs where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

To guard against the race condition, we need to ensure that only one process at a time can be manipulating the variable count.

The Critical Section Problem

Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be accessing and updating data that is shared with at least one other process.

The important feature is when one process is executing in its critical section, no other process is allowed to execute in its critical section.

The **critical-section problem** is to design a **protocol** that the processes can use to synchronize their activity so as to cooperatively share data.

The Critical Section Problem

Each process must **request permission** to enter its critical section.

The section of code implementing this request is the **entry section**.

The critical section may be followed by an **exit section**.

The remaining code is the **remainder section**.

Elements/Sections of a Program/Process

- **Entry Section:** The entry Section decides the entry of a process.
- **Critical Section:** The Critical section allows and makes sure that only one process is modifying the shared data.
- **Exit Section:** The entry of other processes in the shared data after the execution of one process is handled by the Exit section.
- **Remainder Section:** The remaining part of the code which is not categorized as above is contained in the Remainder section.

Critical Section Problem - Primitives

```
while (true) {
```

entry section

critical section

exit section

remainder section

```
}
```

Requirements for Synchronization

A solution to the critical-section problem must satisfy the following three requirements:

- **Mutual exclusion:** If a process is running in the critical section, no other process should be allowed to run in that section at that time.
- **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

Requirements for Synchronization

- **Bounded waiting.** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Race Condition

- When more than one process is either running the same code or modifying the same memory or any shared data, there is a risk that the result or value of the shared data may be incorrect because all processes try to access and modify this shared resource. This condition is called the race condition.

Race Condition

- Since many processes use the same data, the results of the processes may depend on the **order of their execution** (Sequence of their execution)
- In the critical section, a race condition occurs when the end result of multiple thread executions varies depending on the **sequence** in which the threads execute.

Race Condition Example

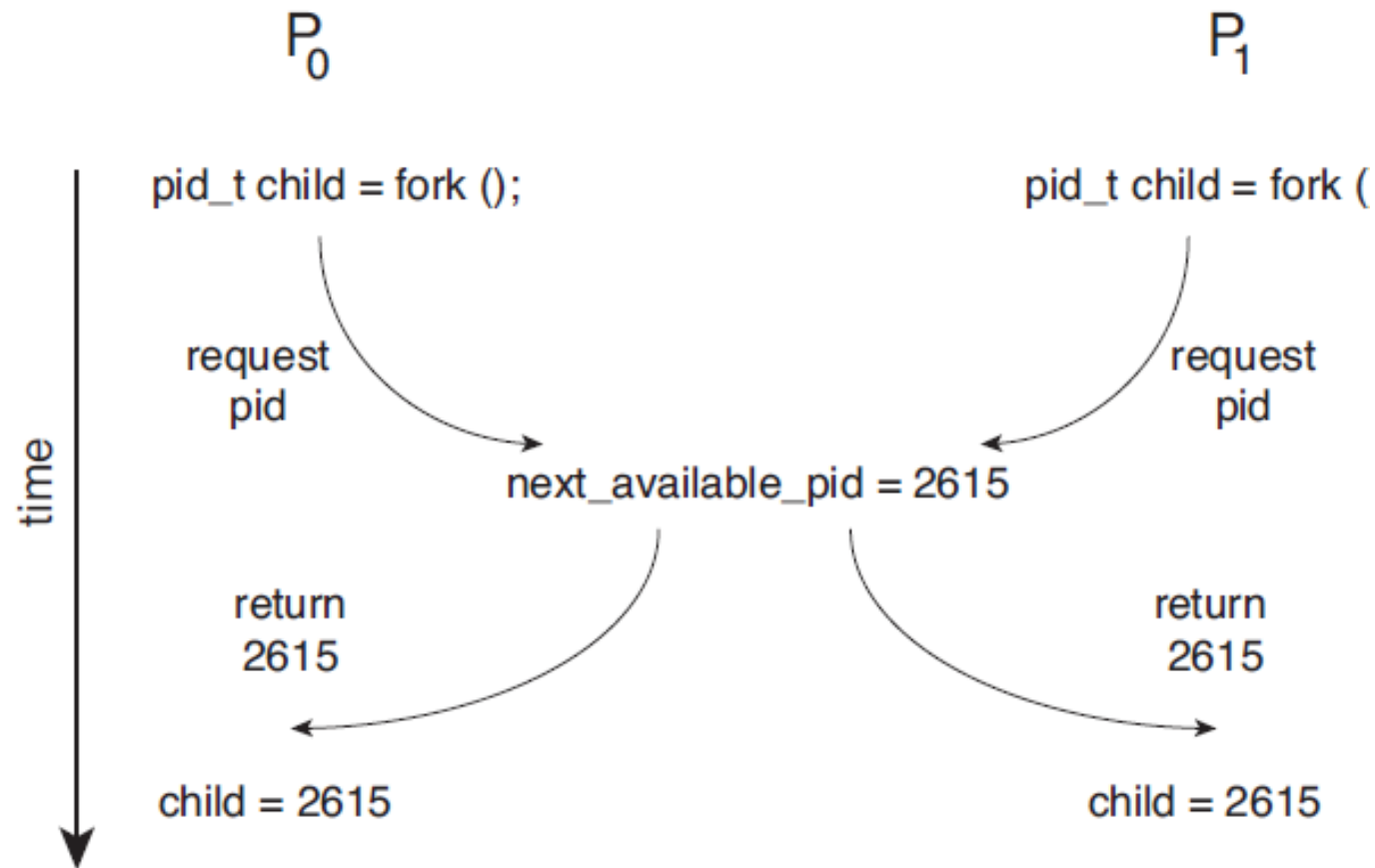


Figure: Race condition when assigning a pid

Critical Section Problem

- The critical-section problem could be solved simply in a **single-core environment** if we could **prevent interrupts** from occurring while a shared variable was being modified.
- This solution is not as feasible in a multiprocessor environment.

Critical Section Problem

- **Two general approaches** are used to handle critical sections in operating systems: preemptive kernels and nonpreemptive kernels.
- **A preemptive kernel** allows a process to be preempted while it is running in kernel mode.
- **A nonpreemptive kernel** does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

Critical Section Problem

- A nonpreemptive kernel is essentially free from race conditions on kernel data structures.
- A preemptive kernel may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes.

Critical Section Problem – What is the Problem?

- A critical section is a code segment that can be accessed by only one process at a time. (How to do it?)
- Designing a way for cooperative processes to access shared resources without creating data inconsistencies.

Critical Section Problem – Peterson's solution

- a classical *software-based solution*.
- This solution makes sure that only one process executes the critical section at any point in time.
- It is restricted to two process that alternate execution between the critical execution and remainder execution.

Critical Section Problem – Peterson's solution

Peterson's solution

Two shared variables that are used by the processes.

- **A boolean flag[]:** A Boolean array Flag indicates if a process is ready to enter into its critical section
- **int turn:** indicates whose turn it is to enter into critical section.

Critical Section Problem – Peterson's solution

Peterson's solution

// Shared variables

bool flag[2] = {false, false};

int turn = 0;

// Initially both flags are false

// Initially turn is 0 (P1's turn)

Critical Section Problem – Peterson's solution

`int turn`

→ Indicates whose turn it is to enter its critical section.

Structure of process P_i in Peterson's solution

do {

```
flag [i] = true ;
turn = j ;
while ( flag [ j ] && turn == [ j ] ) ;
```

critical section

```
flag [i] = false ;
```

remainder section

```
} while (TRUE) ;
```

`boolean flag [2]`

→ Used to indicate if a process is ready to enter its critical section.

Structure of process P_j in Peterson's solution

do {

```
flag [ j ] = true ;
turn = i ;
while ( flag [ i ] && turn == [ i ] ) ;
```

critical section

```
flag [ j ] = false ;
```

remainder section

```
} while (TRUE) ;
```

Critical Section Problem – Peterson's solution

```

while (true) {
    flag[i] = true;
    turn = j; //givingpriority to Pj (the jth
process) in case Pj is also trying to enter the
critical section.
    while (flag[j] && turn == j) {
        // Busy wait until process j's flag is false
or turn is i
    }
    // Critical Section (Pi)
    flag[i] = false; // Process i has exited CS

```

```

while (true) {
    flag[j] = true;
    turn = i;
    while (flag[i] && turn == i) {
        // Busy wait until process i's flag
is false or turn is j
    }
    // Critical Section (Pj)
    flag[j] = false; // Process j has exited CS

```

Critical Section Problem – Peterson's solution

- Each process has a flag variable (e.g., flag[0] for P0 and flag[1] for P1).
- The flag[i] is set to true when process P_i wants to enter the critical section, signaling its intent.
- A turn variable is used to decide which process should proceed if both processes are trying to enter the critical section simultaneously.
- This ensures that the other process gets a fair chance to access the critical section.

Critical Section Problem – Peterson's solution

- Before Entering the Critical Section:
 - A process P_i sets $\text{flag}[i] = \text{true}$, indicating it wants to enter the critical section.
 - It then sets $\text{turn} = j$, where j is the index of the other process (e.g., if $i = 0$, then $j = 1$).
 - The process then enters a busy-wait loop: `while (flag[j] && turn == j)`.

Critical Section Problem

- Before Entering the Critical Section:
 - This loop will only exit if the other process P_j is not interested in entering the critical section ($\text{flag}[j] = \text{false}$) or if the current process P_i is allowed to enter because the turn is in its favor ($\text{turn} \neq j$).
- Critical Section:
 - Once the loop exits, process P_i enters the critical section. At this point, P_i can safely access shared resources without interference from P_j .

Critical Section Problem

- Exiting Critical Section:
 - After finishing the work in the critical section, process P_i sets $\text{flag}[i] = \text{false}$, indicating it no longer needs access to the critical section. This action allows the other process P_j to proceed if it is waiting.

Critical Section Problem

- Boolean flag and turn are both shared variable
- Initially, the flags are false.
- When a process wants to execute its critical section, it sets its
 - flag to true and
 - turn into the index of the other process
 - This means that the process wants to execute but it will allow the other process to run first.

Critical Section Problem

- Set `flag[i]` to true to indicate its intention to enter the critical section.
- Set `turn` to `j` to yield the right to enter the critical section to process `j`.
- Enter a busy waiting loop, checking if `flag[j]` is true and if `turn` is `j`. If both conditions are met, the process continues waiting.
- Once the conditions are no longer met, the process enters its critical section.
- After exiting the critical section, set `flag[i]` to false to indicate that it has finished.

Critical Section Problem

Peterson's solution

- The process performs **busy waiting** until the other process has finished its own critical section.
- After this, the current process enters its critical section
- After completing the critical section, it sets its own flag to false, indicating it does not wish to execute anymore

Critical Section Problem

Peterson's solution

Busy Waiting:

- Busy waiting is a technique in which a process repeatedly checks a condition to enter its critical section instead of being blocked.
- Busy waiting occurs in the while loops where each process checks the flag and turn variables to determine if it's safe to enter its critical section.
- During busy waiting, the process consumes CPU cycles while it waits for the condition to become true.

Critical Section Problem

Peterson's solution - Points

- The turn variable acts as a tie-breaker when both processes want to enter the critical section at the same time.
- If both processes set their respective flag to true (indicating their intent to enter the critical section), the turn variable helps to decide which process gets to enter first.

Critical Section Problem – Bakery Algorithm

Critical Section Problem – Bakery Algorithm

Bakery Algorithm

- Designed to ensure that only one process can access a critical section at a time.
- It is suitable when we have many number of process i.e. $n > 2$
- It's particularly useful in distributed systems where processes might be running on different machines.

Critical Section Problem

Bakery Algorithm

- Each process is assigned a unique ticket number. These numbers are ordered lexicographically, meaning they are compared based on their individual digits from left to right.
- Before entering the critical section, a process acquires a ticket number. This number is determined by finding the maximum ticket number among all other processes and then incrementing it.

Critical Section Problem

Bakery Algorithm

- A process waits until its ticket number is the smallest among all processes that are currently trying to enter the critical section or have already entered.
- Once a process has the smallest ticket number, it can enter the critical section.
- After exiting the critical section, the process releases its ticket, allowing other processes to acquire tickets and enter the critical section.

Critical Section Problem

Bakery Algorithm - Example

Consider three processes, P1, P2, and P3.

- Initial State: All processes have their tickets set to 0.
- P1 Acquires Ticket: P1 acquires ticket number 1.
- P2 Acquires Ticket: P2 acquires ticket number 2.
- P3 Acquires Ticket: P3 acquires ticket number 3.

Critical Section Problem

Bakery Algorithm - Example

- Now, P1 has the smallest ticket number and can enter the critical section.
- After P1 finishes and releases its ticket,
- P2 has the smallest ticket number and can enter.
- Finally, P3 will enter.

Bakery Algorithm

Simplified Bakery Algorithm (example)

- Processes numbered 0 to N-1
- num is an array N integers (initially 0).
 - Each entry corresponds to a process

```
lock(i){  
    num[i] = MAX(num[0], num[1], ....., num[N-1]) + 1  
    for(p = 0; p < N; ++p){  
        while (num[p] != 0 and num[p] < num[i]);  
    }  
}
```

critical section

```
unlock(i){  
    num[i] = 0;  
}
```

P1	P2	P3	P4	P5
0	0	0	0	0

Bakery Algorithm

Original Bakery Algorithm (example)

- Without atomic operation assumptions
- Introduce an array of N Booleans: *choosing*, initially all values False.

```
lock(i){  
    choosing[i] = True  
    num[i] = MAX(num[0], num[1], ..., num[N-1]) + 1  
    choosing[i] = False  
    for(p = 0; p < N; ++p){  
        while (choosing[p]);  
        while (num[p] != 0 and (num[p],p)<(num[i],i));  
    }  
}
```

doorway

critical section

```
unlock(i){  
    num[i] = 0;  
}
```

P1	P2	P3	P4	P5
0	3	1	2	2

$(a, b) < (c, d)$ which is equivalent to: $(a < c)$ or $((a == c) \text{ and } (b < d))$

Critical Section Problem – Bakery Algorithm

repeat

 choosing[i] := true; _____

 number[i] := max(number[0], number[1], ..., number[n-1]) + 1; _____

 choosing[i] := false; _____

 for j := 0 to n - 1 do

 begin

 while choosing[j] do no-op; // Wait until process j finishes choosing its number

 while number[j] != 0 and (number[j], j) < (number[i], i) do no-op; //

Wait until it's i's turn

 end;

 // Critical section

 number[i] := 0; // Exit the critical section

 // Remainder section

until false;

Critical Section Problem

Bakery Algorithm - Points

- **Fairness:** It ensures that all processes eventually get to enter the critical section, preventing starvation.
- **Freedom from Starvation:** No process is indefinitely prevented from entering the critical section.
- **Freedom from Deadlock:** The algorithm prevents deadlocks by ensuring that processes always make progress.
- **Distributed:** It can be used in distributed systems where processes may be running on different machines.

Critical Section Problem

Bakery Algorithm - Points

- Choosing Array (choosing[i]):
 - This is a boolean array where choosing[i] is set to true when process i is selecting a number (a "ticket") for itself before entering the critical section. It is set to false once the process has chosen its number

Critical Section Problem

Bakery Algorithm - Points

- Number Array (number[i]):
 - This array holds the number (or "ticket") for each process. The number represents the process's place in line to enter the critical section.
 - A higher number means the process will wait longer, and a lower number means it gets to enter sooner.

Critical Section Problem

Bakery Algorithm - Points

- **Choosing the number or Ticket**
- `choosing[i] := true;` Process *i* sets its choosing flag to true indicating that it's in the process of choosing a number.
- `Number[i] := max(number[0], number[1], ..., number[n-1]) + 1;` Process *i* picks a number that is one greater than the maximum number in the number array. This step ensures that each process gets a unique and sequential number.

Critical Section Problem

Bakery Algorithm - Points

- `choosing[i] := false`;: Process *i* sets its choosing flag to false after it has chosen its number.

Critical Section Problem

Bakery Algorithm - Points

- **Waiting to Enter the Critical Section:**
- for $j := 0$ to $n - 1$: Process i checks all other processes j .
- while choosing[j] do no-op;: Process i waits until process j finishes choosing its number. This ensures that if j is in the middle of choosing its number, i does not make a decision based on incomplete information.

Critical Section Problem

Bakery Algorithm - Points

- **Waiting to Enter the Critical Section:**
- while $\text{number}[j] \neq 0$ and $(\text{number}[j], j) < (\text{number}[i], i)$ do no-op;:
 - If process j has a lower number (indicating it has higher priority) and j has not yet entered the critical section ($\text{number}[j] \neq 0$), process i will wait.
 - If $\text{number}[j] == \text{number}[i]$, the process with the smaller index j gets priority. This ensures fairness and prevents deadlock.

Critical Section Problem

Bakery Algorithm - Points

▪ Entering the Critical Section:

- Once process i has confirmed that it has the smallest number (or highest priority) among all competing processes, it proceeds to enter the critical section.

▪ Exiting the Critical Section:

- $\text{number}[i] := 0$;: After finishing in the critical section, process i resets its number to 0, indicating that it no longer wishes to enter the critical section.

Critical Section Problem

Bakery Algorithm - Points

- Remainder Section:
 - The process can then perform other non-critical operations before repeating the process if it needs to enter the critical section again.

Critical Section Problem

Bakery Algorithm - Points

- Simplest known solutions to the mutual exclusion problem for n processes.
- Ensures that shared resources are used efficiently in a multithreaded environment.
- It is free from starvation.
- It uses FIFO
- It works with atomic registers.

Critical Section Problem

Bakery Algorithm - Points

- It is unreliable because any one of the processes can fail and halt progress.